

RASPBERRY PI TELEGRAM BOT ANLEITUNG

Deutsche Ausgabe // einsteigerfreundlicher Bot-Knoten

Build

- Telegram BotFather
- Python bot
- SQLite notes
- PuTTY workflow

Operate

- systemd autostart
- restart after crash
- reboot recovery
- backup and restore

Light corporate cyberpunk edition // v1.2 // copy/paste-first tutorial for beginners.

WAS AM ENDE LAUFEN SOLL

Ziel der Anleitung

Ein zuverlässiger Telegram-Bot auf einem Raspberry Pi: BotFather-Token, Python-Bot, SQLite-Notizen, Backup-Dateien, PuTTY-Workflow, systemd-Autostart, Reboot-Recovery und optionale lokale KI. Die Anleitung nutzt komplette Copy/Paste-Blöcke.

Ebene	Auswahl	Warum
Telegram	BotFather-Token	Offizieller Weg zur Bot-Erstellung
Host	Raspberry Pi 3/4/5	Sparsamer Always-on-Linux-Knoten
Runtime	Python venv	Saubere Abhängigkeiten
Service	systemd	Autostart und Neustart nach Fehlern
Storage	SQLite	Einfache lokale Datenbank
KI optional	Ollama	Erst nach stabilem Basis-Bot

COPY/PASTE OHNE CHAOS

So benutzt du die Anleitung

Arbeite in Etappen. Einen Block einfügen, Ausgabe lesen, dann weiter. In PuTTY sollte das Fenster bei Fehlern offen bleiben.

Regel 1

Bei Fehlern stoppen und Logs lesen.

Regel 2

Echten Token niemals in GitHub posten. Nur in .env speichern.

Regel 3

Bei Änderungen: sichern, kompilieren, Service neu starten.

PUTTY EINRICHTEN

```
# Windows: download PuTTY from the official PuTTY website.
# Start PuTTY, then fill in:
Host Name: raspberrypi.local
Port: 22
Connection type: SSH

# Recommended PuTTY setting:
Window -> Behaviour -> Close window on exit -> Never
Session -> Save
```

Hardware-Uebersicht

Ein normaler Telegram-Bot ist leicht. Lokale KI ist schwer.

Host	Bot	Local AI	Comment
Pi 3 / 3 B+	Yes	No LLM	Commands, notes, sensors
Pi 4 2GB	Yes	No LLM / 0.5B only	Service node
Pi 4 4GB	Yes	0.5B Q4, 1.5B slow	Short prompts
Pi 4 8GB	Yes	0.5B-1.5B, 3B experimental	Best Pi 4
Pi 5 4GB	Strong	0.5B-1.5B	Fast bot
Pi 5 8GB	Recommended	1.5B-3B	Best normal choice
Pi 5 16GB	Best Pi	3B-4B, 7B slow	Most RAM

OLLAMA UND HUIHUI-BEISPIELE

Lokale KI einschuetzen

Ein Telegram-Bot soll schnell antworten. Lokale KI besser hinter Befehle legen.

Model size	Pi 4 4GB	Pi 4 8GB	Pi 5 8GB	Pi 5 16GB
0.5B Q4	Usable	Usable	Smooth	Smooth
1.5B Q4	Slow	Usable	Good	Good
3B Q4	Avoid	Slow	Usable	Good
4B Q4/Q5	Avoid	Avoid/slow	Borderline	Usable
7B Q4	Avoid	Avoid	Slow	Possible

OPTIONALE OLLAMA-TESTS

```
# Optional. Do this only after the base bot is reliable.
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
ollama run qwen2.5:0.5b
ollama run qwen2.5:1.5b
ollama run qwen2.5:3b

# Huihui examples - check current tags on ollama.com/huihui_ai first.
ollama run huihui_ai/phi4-mini-abliterated
ollama run huihui_ai/qwen3-abliterated
```

Aktuelle Modell-Tags auf Ollama vor dem Veröffentlichen pruefen.

TELEGRAM-BOT ERSTELLEN

BotFather

In Telegram @BotFather suchen, neuen Bot erstellen und Token spaeter in .env eintragen.

BOTFATHER-BEFEHLE

```
/newbot

# Display name example:
Raspi Node Bot

# Username example. It must end in bot:
raspi_node_demo_bot

/setcommands
start - boot message
status - service health
last - show recent notes
files - export backup files
help - command list
```

HEADLESS-GRUNDLAGE

Raspberry Pi OS flashen

Mit Raspberry Pi Imager. Raspberry Pi OS Lite 64-bit reicht.

ERSTER LOGIN

```
# Windows PowerShell / macOS Terminal / Linux Terminal:
ssh pi@raspberrypi.local

# If .local does not resolve, use the IP address from your router:
ssh pi@192.168.1.50
```

PI ERREICHBAR?

Erster Check

Nach dem SSH-Login kurz Identitaet, IP und Uptime pruefen.

IDENTITY CHECK

```
whoami
hostname
hostname -I
uname -a
uptime
```

PYTHON, VENV, PROJEKTORDNER

Basis installieren

Erstellt ~/telegram-bot und installiert Python-Abhängigkeiten.

PAKETE UND VENV

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y python3-venv python3-pip sqlite3 git curl nano ca-certificates

mkdir -p ~/telegram-bot
cd ~/telegram-bot
python3 -m venv venv
./venv/bin/python -m pip install --upgrade pip
./venv/bin/pip install python-telegram-bot==22.8
```

TOKEN OHNE HARDCODING

.env erstellen

Der Bot liest BOT_TOKEN aus .env.

CREATE .ENV

```
cd ~/telegram-bot
cat > .env <<'ENVFILE'
BOT_TOKEN=PASTE_BOTFATHER_TOKEN_HERE
BOT_NAME=raspi-node
ENVFILE

chmod 600 .env
ls -l .env
```

Allgemeiner Bot-Code

Generischer Code: normale Nachrichten werden sofort gespeichert.

BOT.PY SCHREIBEN

```
cd ~/telegram-bot
cat > bot.py <<'BOTPY'
import json
import os
import re
import sqlite3
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo

from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, ContextTypes, filters

BASE = Path(__file__).resolve().parent
DB_FILE = BASE / "bot.db"
ENV_FILE = BASE / ".env"
BACKUP_DIR = BASE / "backups"
TZ = ZoneInfo("Europe/Berlin")

def load_env():
    if not ENV_FILE.exists():
        return
    for line in ENV_FILE.read_text(encoding="utf-8", errors="ignore").splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))

def now_local():
    return datetime.now(TZ)

def clean_text(text):
    text = str(text or "").strip()
    text = re.sub(r"\s+", " ", text)
    for before, after in {" ": " ", ".": ".", " !": "!", " ?": "?"}.items():
        text = text.replace(before, after)
    return text.strip()

def db():
    con = sqlite3.connect(DB_FILE)
    con.row_factory = sqlite3.Row
    return con

def init_db():
    con = db()
    cur = con.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            created_at TEXT NOT NULL,
            text TEXT NOT NULL
        )
    """)
    con.commit()
    con.close()

def add_note(text):
    text = clean_text(text)
    if not text:
        raise ValueError("empty note")
    con = db()
    cur = con.cursor()
    cur.execute("INSERT INTO notes (created_at, text) VALUES (?, ?)",
        (now_local().isoformat(timespec="seconds"), text))
    note_id = cur.lastrowid
    con.commit()
    con.close()
    return note_id

def get_last(limit=10):
    con = db()
    rows = con.execute("SELECT id, created_at, text FROM notes ORDER BY id DESC LIMIT ?",
        (limit,)).fetchall()
    con.close()
    return list(reversed(rows))

def format_notes(rows):
    if not rows:
```

```

        return "VAULT.EMPTY // no notes stored"
    return "\n".join(f"- #{r['id']} {r['text']}" for r in rows)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "[RASPI.BOT]\nNODE ONLINE // send normal text to store a note\n\n"
        "/status - health check\n/last - latest notes\n/files - export backups\n/help - command list"
    )

async def status(update: Update, context: ContextTypes.DEFAULT_TYPE):
    con = db()
    total = con.execute("SELECT COUNT(*) AS c FROM notes").fetchone()["c"]
    con.close()
    await update.message.reply_text(f"STATUS // ONLINE\nSTORED // {total} notes")

async def last(update: Update, context: ContextTypes.DEFAULT_TYPE):
    limit = 10
    if context.args:
        try:
            limit = max(1, min(50, int(context.args[0])))
        except ValueError:
            limit = 10
    await update.message.reply_text("[LATEST]\n" + format_notes(get_last(limit)))

async def files(update: Update, context: ContextTypes.DEFAULT_TYPE):
    BACKUP_DIR.mkdir(exist_ok=True)
    stamp = now.local().strftime("%Y%m%d-%H%M%S")
    txt = BACKUP_DIR / f"notes-{stamp}.txt"
    jsonl = BACKUP_DIR / f"notes-{stamp}.jsonl"
    rows = get_last(100000)
    txt.write_text(format_notes(rows) + "\n", encoding="utf-8")
    with jsonl.open("w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(dict(r), ensure_ascii=False) + "\n")
    await update.message.reply_text("FILE.NODE // exporting backup files")
    await update.message.reply_document(document=txt.open("rb"), filename=txt.name)
    await update.message.reply_document(document=jsonl.open("rb"), filename=jsonl.name)

async def help_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await start(update, context)

async def store_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = clean_text(update.message.text)
    if not text:
        await update.message.reply_text("INTAKE.REJECTED // empty message")
        return
    note_id = add_note(text)
    await update.message.reply_text(f"[LOGGED]\n- #{note_id} {text}")

def main():
    load_env()
    token = os.environ.get("BOT_TOKEN", "").strip()
    if not token:
        raise RuntimeError("BOT_TOKEN missing in .env")
    init_db()
    app = Application.builder().token(token).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", help_cmd))
    app.add_handler(CommandHandler("status", status))
    app.add_handler(CommandHandler("last", last))
    app.add_handler(CommandHandler("files", files))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, store_text))
    app.run_polling(allowed_updates=Update.ALL_TYPES, drop_pending_updates=True)

if __name__ == "__main__":
    main()
BOTPY

```

VOR SYSTEMD TESTEN

Manueller Test

Den Bot zuerst manuell starten.

MANUELLER TEST

```
cd ~/telegram-bot
./venv/bin/python -m py_compile bot.py
./venv/bin/python bot.py

# Stop the manual run with Ctrl+C after the Telegram test works.
```

AUTOSTART NACH NEUSTART UND STROMAUSFALL

Als Service installieren

systemd startet den Bot beim Booten und nach Absturz neu.

SERVICE ERSTELLEN UND AKTIVIEREN

```
sudo tee /etc/systemd/system/telegram-bot.service > /dev/null <<'SERVICE'
[Unit]
Description=Raspberry Pi Telegram Bot
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/telegram-bot
EnvironmentFile=/home/pi/telegram-bot/.env
ExecStart=/home/pi/telegram-bot/venv/bin/python /home/pi/telegram-bot/bot.py
Restart=always
RestartSec=10
TimeoutStartSec=60

[Install]
WantedBy=multi-user.target
SERVICE

sudo systemctl daemon-reload
sudo systemctl enable telegram-bot
sudo systemctl start telegram-bot
systemctl status telegram-bot --no-pager -l
```

Wenn dein User nicht pi heisst, User=pi und /home/pi ersetzen.

NACHWEISEN, DASS ER WIEDERKOMMT

Reboot- und Stromausfalltest

Ein Service ist erst vertrauenswürdig, wenn ein Reboot-Test funktioniert.

REBOOT-RECOVERY-TEST

```
# Test normal reboot recovery:
sudo reboot

# Reconnect after 30-90 seconds:
ssh pi@raspberrypi.local

# Check whether the bot came back automatically:
systemctl status telegram-bot --no-pager -l
journalctl -u telegram-bot -n 80 --no-pager
```


WENN ETWAS NICHT LAEUFT

Logs und Reparatur

Nicht raten. status und journalctl nutzen.

SERVICE-LOGS

```
systemctl status telegram-bot --no-pager -l
journalctl -u telegram-bot -n 120 --no-pager
journalctl -u telegram-bot -f
```

FUER EINSTEIGER OHNE NANO

Sicher per Copy/Paste aendern

Service stoppen, Backup, ersetzen oder patchen, kompilieren, neu starten.

KOMPLETTE DATEI SICHER ERSETZEN

```
cd ~/telegram-bot
sudo systemctl stop telegram-bot
cp bot.py bot.py.bak-$(date +%Y%m%d-%H%M%S)
cat > bot.py <<'BOTPY'
# paste the complete new bot.py here
BOTPY
./venv/bin/python -m py_compile bot.py
sudo systemctl start telegram-bot
systemctl status telegram-bot --no-pager -l
```

TEXT AENDERN ODER BEFEHL HINZUFUEGEN

Kleine Code-Aenderungen

Kleine Patches fuer kleine Aenderungen.

Kleiner Patch

Fuer Textaenderungen.

Komplett ersetzen

Sicherer bei groesseren Aenderungen.

ANTWORTTEXT PATCHEN

```
cd ~/telegram-bot
cp bot.py bot.py.bak-text-change-$(date +%Y%m%d-%H%M%S)
python3 - <<'PY'
from pathlib import Path
p = Path('bot.py')
s = p.read_text(encoding='utf-8')
s = s.replace('NODE ONLINE // send normal text to store a note', 'NODE ONLINE // intake ready')
p.write_text(s, encoding='utf-8')
PY
./venv/bin/python -m py_compile bot.py && sudo systemctl restart telegram-bot
```

MINIMALMUSTER FUER COMMANDS

Befehle erweitern

Jeder Befehl braucht async-Funktion plus CommandHandler.

/PING HINZUFUEGEN

```
# Add this function above main():
async def ping(update, context):
    await update.message.reply_text("PONG // node alive")

# Then add this handler inside main(), near the other handlers:
app.add_handler(CommandHandler("ping", ping))

# Check and restart:
./venv/bin/python -m py_compile bot.py && sudo systemctl restart telegram-bot
```

EINER SD-KARTE NICHT BLIND VERTRAUEN

Backup und Restore

Projektordner und Datenbank sichern.

ORDNER SICHERN UND WIEDERHERSTELLEN

```
# Create a local backup archive:
cd ~
tar -czf telegram-bot-backup-$(date +%Y%m%d-%H%M%S).tar.gz telegram-bot
ls -lh telegram-bot-backup-*.tar.gz | tail

# Restore an archive:
cd ~
sudo systemctl stop telegram-bot
mv telegram-bot telegram-bot.bak-$(date +%Y%m%d-%H%M%S)
tar -xzf telegram-bot-backup-YYYYMMDD-HHMMSS.tar.gz
sudo systemctl start telegram-bot
```

SYMPTOME ZU BEFEHLEN

Troubleshooting-Karte

Erst diese Seite nutzen.

Symptom	Likely cause	Command
Bot silent	Service down	systemctl status telegram-bot
Token error	Bad .env	check .env carefully
Delayed replies	Slow handler/network	journalctl -u telegram-bot -f
PuTTY closes	Session exits	Close window on exit = Never
Duplicate replies	Two bots running	ps -ef grep python
Database error	Schema mismatch	sqlite3 bot.db .schema
No autostart	Service not enabled	systemctl is-enabled telegram-bot

VOLLCHECK

```
cd ~/telegram-bot
./venv/bin/python -m py_compile bot.py
systemctl is-enabled telegram-bot
systemctl is-active telegram-bot
systemctl status telegram-bot --no-pager -l | head -n 60
journalctl -u telegram-bot -n 80 --no-pager
```

VOR GITHUB ODER DAUERBETRIEB

Sicherheitscheck

Telegram-Tokens sind Zugangsdaten.

RECHTE PRUEFEN

```
cd ~/telegram-bot
ls -lah
ls -l .env bot.py bot.db 2>/dev/null
chmod 600 .env
chmod 644 bot.py
```

FAQ

Brauche ich einen Raspberry Pi 5?

Nein. Fuer einen einfachen Telegram-Bot reichen Pi 3 oder Pi 4. Pi 5 lohnt sich fuer mehr Automatisierung oder lokale KI-Tests.

Startet der Bot nach Stromausfall wieder?

Ja, wenn der systemd-Service aktiviert ist und der Pi normal bootet.

Warum systemd?

systemd startet den Bot beim Booten, startet ihn nach Abstuerzen neu und liefert Logs ueber journalctl.

Was ist BotFather?

BotFather ist der offizielle Telegram-Bot zum Erstellen und Verwalten eigener Bots.

Wo kommt der Token hin?

In die Datei .env, nie direkt in bot.py und nie in GitHub.

Kann ich PuTTY nutzen?

Ja. Fuer Windows ist PuTTY sehr praktisch. Setze Close window on exit auf Never.

Warum schliesst sich PuTTY?

Meist beendet sich die Shell. Aendere die PuTTY-Einstellung oder nutze Bloecke ohne exit.

Kann ich die Bloecke direkt kopieren?

Ja. Vorher Platzhalter wie BOT_TOKEN und Username ersetzen.

Kann ich ohne nano bearbeiten?

Ja. Nutze komplette heredoc-Dateibloecke oder kleine Python-Patches.

Was wenn der Bot nicht antwortet?

Pruefe systemctl status und journalctl, dann Service neu starten.

Was wenn /today oder /last stark verzoegert ist?

Keine langsame KI im normalen Message-Handler nutzen. Erst speichern, spaeter auswerten.

Kann lokale KI jede Notiz klassifizieren?

Technisch ja, praktisch auf dem Pi oft zu langsam. Besser per explizitem Befehl.

Welche Datenbank wird genutzt?

SQLite. Einfach, lokal, ohne extra Server.

Kann ich den Bot oeffentlich betreiben?

Der Bot ist ueber Telegram erreichbar. SSH nicht offen ins Internet legen, wenn du es nicht absichern kannst.

Kann ich mehrere Bots betreiben?

Ja, aber mit getrennten Ordnern, Services und BotFather-Tokens.

Was passiert bei zwei Prozessen mit gleichem Token?

Es kann Polling-Konflikte geben. Doppelte Services stoppen.

Wie finde ich laufende Python-Bots?

Mit `ps -ef | grep -i python` und `systemctl list-units`.

Wie update ich Code sicher?

Service stoppen, Datei sichern, Code ersetzen, `py_compile`, Service starten.

Was ist drop_pending_updates?

Alte Telegram-Updates werden beim Start verworfen.

Kann der Bot Dateien senden?

Ja. Das Beispiel /files exportiert Backups als TXT und JSONL.

Kann ich Backups auf dem Pi behalten?

Ja, aber zusaetzlich extern kopieren. SD-Karten fallen aus.

Soll ich eine SSD nutzen?

Fuer dauerhaften Betrieb ist SSD robuster als billige microSD.

Ist WLAN okay?

Ja, fuer viele Bots. Ethernet ist stabiler.

Garantiert network-online.target dauerhaft Internet?

Nein. Es wartet nur beim Start. Spaetere Netzabbrueche muss der Bot vertragen.

Was wenn Telegram nicht erreichbar ist?

Der Service startet neu und sollte sich erholen, sobald das Netz wieder da ist.

Welcher Pi ist fuer lokale KI am besten?

Pi 5 mit 8GB oder 16GB. Trotzdem bleibt es CPU-limitiert.

Welche Modelle sind realistisch?

0.5B bis 3B quantisiert sind praktisch. 4B geht eher auf Pi 5 16GB. 7B ist nicht flüssig.

Kann ich Huihui-Modelle nutzen?

Ja, aber aktuelle Tags prüfen und klein testen.
Abliterated-Modelle privat und beaufsichtigt nutzen.

Brauche ich Docker?

Nein. Diese Anleitung nutzt Python direkt und systemd.

Geht Raspberry Pi OS Desktop?

Ja, aber Lite braucht weniger Ressourcen.

Wie ändere ich den Bot-Namen?

In BotFather ändern und Text in bot.py anpassen.

Wie füge ich Befehle hinzu?

Async-Funktion schreiben und mit CommandHandler registrieren.

Warum py_compile?

Es findet Syntaxfehler vor dem Neustart.

Welche Lizenz passt?

Wenn jeder alles frei nutzen darf, ist CC0 am einfachsten.

Kann ich das Repo veröffentlichen?

Ja, solange keine Tokens, Datenbanken, Chat-IDs oder privaten Logs enthalten sind.

Was teste ich am Ende?

Manueller Start, Telegram-Antwort, systemd-Start, Reboot, Logs und Backup-Export.

Quellen und Referenzen

Telegram Bot tutorial / BotFather

<https://core.telegram.org/bots/tutorial>

BotFather profile and management entry

<https://telegram.me/BotFather>

python-telegram-bot documentation

<https://docs.python-telegram-bot.org/en/stable/>

Raspberry Pi SSH remote access documentation

<https://www.raspberrypi.com/documentation/configuration/ssh/remote-access.html>

Raspberry Pi 3 Model B+ product page

<https://www.raspberrypi.com/products/raspberrypi-3-model-b-plus/>

Raspberry Pi 4 Model B specifications

<https://www.raspberrypi.com/products/raspberrypi-4-model-b/specifications/>

Raspberry Pi 5 product page

<https://www.raspberrypi.com/products/raspberrypi-5/>

systemd network-online guidance

https://systemd.io/NETWORK_ONLINE/

Ollama documentation

<https://docs.ollama.com/>

Ollama Huihui phi4-mini abliterated model page

https://ollama.com/huihui_ai/phi4-mini-abliterated

Ollama Huihui qwen3 abliterated model page

https://ollama.com/huihui_ai/qwen3-abliterated